

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
KHOA CÔNG NGHỆ THÔNG TIN

LCMFreq

Khai thác tập phổ biến bằng Backtracking, OccurrenceDeliver và Hypercube

Sinh viên thực hiện

STT	Họ và tên	MSSV
1	Phạm Phú Hòa (nhóm trưởng)	23122030
2	Huỳnh Trung Kiệt	23122039
3	Đào Sỹ Duy Minh	23122041
4	Trần Chí Nguyên	23122044
5	Nguyễn Lâm Phú Quý	23122048

Giảng viên hướng dẫn

GS.TS. Lê Hoài Bắc
ThS. Nguyễn Thị Thu Hằng
ThS. Nguyễn Ngọc Đức
ThS. Lê Nhật Nam

Môn học

CSC14004, Khai thác dữ liệu và ứng dụng

Đặt vấn đề

Khung Backtracking

Đếm support hiệu quả

HypercubeDecomposition

Pseudo-code và ví dụ chạy tay

Cài đặt Julia

Đánh giá thực nghiệm

Ứng dụng: phân tích giỏ hàng

Kết luận

Tài liệu tham khảo

Đặt vấn đề

Khung Backtracking

Đếm support hiệu quả

HypercubeDecomposition

Pseudo-code và ví dụ chạy tay

Cài đặt Julia

Đánh giá thực nghiệm

Ứng dụng: phân tích giỏ hàng

Kết luận

Tài liệu tham khảo

Transaction database và itemset

Gọi $\mathcal{I} = \{1, \dots, n\}$ là tập item. Một transaction database trên $\mathcal{I}$ là tập $\mathcal{D} = \{T_1, \dots, T_m\}$, mỗi T_i là một tập con của $\mathcal{I}$, gọi là một giao dịch. Một tập con $P \subseteq \mathcal{I}$ được gọi là một itemset.

Denotation và frequency

Với itemset P , một giao dịch chứa P được gọi là một occurrence của P . Tập tất cả occurrence của P được gọi là denotation của P , ký hiệu $\mathcal{T}(P)$. Số lượng $|\mathcal{T}(P)|$ gọi là frequency của P (trong nhiều tài liệu khác gọi là support), ký hiệu $\text{sup}(P)$. Với ngưỡng tối thiểu σ cho trước, itemset P là **frequent** nếu $\text{sup}(P) \geq \sigma$.

Hai tính chất nền tảng

Với hai itemset P, Q bất kỳ: $\mathcal{T}(P \cup Q) = \mathcal{T}(P) \cap \mathcal{T}(Q)$. Và nếu $P \subseteq Q$ thì $\mathcal{T}(Q) \subseteq \mathcal{T}(P)$, do đó $\text{sup}(Q) \leq \text{sup}(P)$. Bài toán Frequent Itemset Mining yêu cầu tìm toàn bộ itemset frequent của một database cho trước, không bỏ sót.

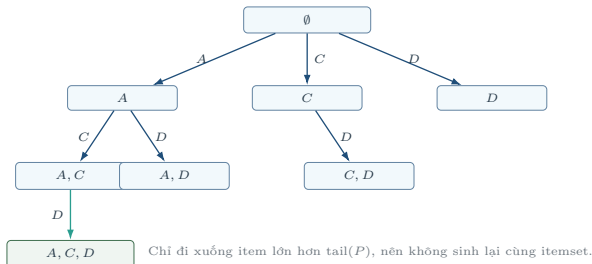
Tính đơn điệu và thứ tự sinh itemset

Tính chất đơn điệu (monotone property)

Từ $P \subseteq Q \Rightarrow \mathcal{T}(Q) \subseteq \mathcal{T}(P)$, suy ra nếu P không frequent thì mọi superset của P cũng không frequent. Nhờ tính chất này, mọi frequent itemset đều có thể được xây dựng từ tập rỗng bằng cách thêm item lần lượt, mà không cần đi qua một itemset không frequent nào ở giữa.

tail(P) và quy tắc sinh không trùng

Gọi $\text{tail}(P)$ là item có chỉ số lớn nhất trong P . Khung backtracking chỉ thêm vào P những item $e > \text{tail}(P)$. Nếu không có quy tắc này, một itemset như $\{A, C, D\}$ có thể sinh ra từ nhiều thứ tự thêm khác nhau và bị tính trùng. Với quy tắc trên, mỗi itemset chỉ có đúng một đường sinh duy nhất.



Đặt vấn đề

Khung Backtracking

Đếm support hiệu quả

HypercubeDecomposition

Pseudo-code và ví dụ chạy tay

Cài đặt Julia

Đánh giá thực nghiệm

Ứng dụng: phân tích giỏ hàng

Kết luận

Tài liệu tham khảo

Nhóm Apriori (level by level)

Gọi $\mathcal{D}_k$ là tập frequent itemset kích thước k . Apriori tính $\mathcal{D}_k$ từ $\mathcal{D}_{k-1}$ theo thứ tự k tăng dần, bằng cách thêm mỗi item vào từng itemset của $\mathcal{D}_{k-1}$ rồi giữ lại các itemset frequent. Toàn bộ $\mathcal{D}_{k-1}$ phải được lưu trong bộ nhớ để sinh $\mathcal{D}_k$.

Nhóm Backtracking

Mỗi lượt đệ quy nhận một frequent itemset P (nghiệm hiện tại), thêm từng item vào P , rồi đệ quy tiếp với các itemset con frequent. Chỉ nghiệm hiện tại được giữ trong bộ nhớ, không cần lưu lại các itemset đã sinh trước đó.

Theo Uno et al. (2004): “Apriori algorithms use much memory for storing $\mathcal{D}_k$, while backtracking algorithms use less memory since they keep only the current solution... However, apriori algorithms have advantages for the frequency counting.” Đây chính là động cơ của LCMFreq: chọn khung backtracking để tiết kiệm bộ nhớ, đồng thời khắc phục điểm yếu của backtracking trong việc đếm support, bằng hai kỹ thuật trình bày ở các phần sau.

```
BackTracking(P: nghiệm hiện tại):  
    output P  
    for e in I, e > tail(P):  
        if P union {e} là frequent:  
            BackTracking(P union {e})
```

Mỗi lượt gọi xuất ra itemset P hiện tại, rồi thử thêm từng item e lớn hơn $\text{tail}(P)$. Nếu $P \cup \{e\}$ frequent thì gọi đệ quy tiếp với $P \cup \{e\}$.

Khung này tạo ra một cây tìm kiếm: mỗi node là một lượt gọi, mỗi cạnh ứng với việc thêm một item.

LCMFreq giữ đúng khung này, và chỉ thay đổi cách thực hiện hai việc bên trong: đếm support nhanh hơn (phần OccurrenceDeliver), và xuất kết quả theo lô (phần HypercubeDecomposition).

1. Backtracking

Mỗi node trong cây tìm kiếm là một prefix P . Chỉ thêm item $e > \text{tail}(P)$, để mỗi itemset có đúng một đường sinh.

2. OccurrenceDeliver

Tại node P , duyệt các giao dịch trong $\mathcal{T}(P)$ một lần, rồi đưa từng giao dịch vào bucket của mọi item nó chứa.

3. Hypercube

Nếu thêm e không làm giảm denotation, mọi tổ hợp chứa các item như vậy đều cùng support, và có thể xuất ra ngay một lượt.



Deliver để biết support của mọi candidate cùng lúc, sau đó lọc ra các item miễn phí (không giảm support), xuất tất cả tổ hợp của các item miễn phí đó trong một lần, rồi mới đệ quy tiếp với các item còn lại.

Đặt vấn đề

Khung Backtracking

Đếm support hiệu quả

HypercubeDecomposition

Pseudo-code và ví dụ chạy tay

Cài đặt Julia

Đánh giá thực nghiệm

Ứng dụng: phân tích giỏ hàng

Kết luận

Tài liệu tham khảo

Down project: cách tổng quát để tính denotation

Định nghĩa down project

Với itemset $P = P_1 \cup P_2$, ta có $\mathcal{T}(P) = \mathcal{T}(P_1) \cap \mathcal{T}(P_2)$. Nếu các item trong $\mathcal{T}(P_1)$ và $\mathcal{T}(P_2)$ đã được sắp theo chỉ số, phép giao này tính được trong $O(|\mathcal{T}(P_1)| + |\mathcal{T}(P_2)|)$, bằng cách đi hai con trỏ song song qua hai danh sách đã sort.

Vấn đề khi áp dụng cho backtracking

Down project hiệu quả nếu chọn được cặp P_1, P_2 tốt, điều Apriori có thể làm. Nhưng backtracking luôn thêm đúng một item mỗi lượt, nên cặp tách duy nhất có thể dùng là $P_1 = P, P_2 = \{e\}$. Với k candidate $e > \text{tail}(P)$, chi phí tính theo down project là:

$$O\left(\sum_{e > \text{tail}(P)} (|\mathcal{T}(P)| + |\mathcal{T}(\{e\})|)\right).$$

Số hạng $|\mathcal{T}(P)|$ bị cộng lặp lại cho từng candidate e , dù $\mathcal{T}(P)$ không đổi giữa các lần đó.

Vì sao OccurrenceDeliver nhanh hơn: ví dụ đếm bước

Xét node $P = \{A\}$, với $\mathcal{T}(P) = \{t_1, t_3, t_5\}$ là ba giao dịch đang chứa A , trong đó $t_1 = \{A, B, C\}$, $t_3 = \{A, C, D\}$, $t_5 = \{A, B, D\}$. Giả sử $\mathcal{T}(B), \mathcal{T}(C), \mathcal{T}(D)$ đã có sẵn từ lượt quét database lúc đầu, với độ dài lần lượt là 5, 4, 3.

Down project: tính riêng từng candidate

Theo công thức ở slide trước:

- $\mathcal{T}(P) \cap \mathcal{T}(B)$: $3 + 5 = 8$ bước
- $\mathcal{T}(P) \cap \mathcal{T}(C)$: $3 + 4 = 7$ bước
- $\mathcal{T}(P) \cap \mathcal{T}(D)$: $3 + 3 = 6$ bước

Tổng **21 bước**. $\mathcal{T}(P)$ (3 phần tử) bị đọc lại **3 lần**, một lần cho mỗi candidate.

OccurrenceDeliver: tính một lần cho cả ba

Duyệt ba giao dịch đúng một lần, mỗi giao dịch tốn bằng số item nó có ngoài A :

- t_1 có B, C : 2 bước
- t_3 có C, D : 2 bước
- t_5 có B, D : 2 bước

Tổng **6 bước**, không cần đọc $\mathcal{T}(B), \mathcal{T}(C), \mathcal{T}(D)$.

Hai mươi một bước so với sáu bước, cho cùng một node. Sau lượt deliver, $\text{Bucket}[e] = \mathcal{T}(P \cup \{e\})$ có sẵn cho cả ba item cùng lúc.

```
OccurrenceDeliver(D: database, P: itemset):  
    for e > tail(P): Bucket[e] := empty  
  
    for t in T(P):  
        for e in t, e > tail(P):  
            insert t into Bucket[e]  
  
    # sau khi ket thuc:  
    # Bucket[e] = T(P union {e})
```

Với mỗi giao dịch t trong $\mathcal{T}(P)$, chi phí xét t chỉ phụ thuộc vào số item của t lớn hơn $\text{tail}(P)$. Tổng chi phí cho cả lượt deliver:

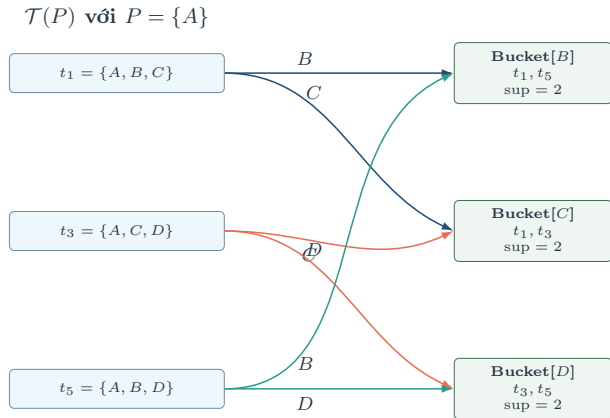
$$O\left(|\mathcal{T}(P)| + \sum_{e > \text{tail}(P)} |\mathcal{T}(P \cup \{e\})|\right).$$

Theo Uno et al. (2004): “*This time complexity is smaller than down project.*” Lý do đúng như ví dụ ở slide trước: số hạng $|\mathcal{T}(P)|$ chỉ xuất hiện một lần trong công thức này, thay vì bị nhân với số candidate.

Minh họa: phân phối giao dịch vào bucket

Xét node $P = \{A\}$ với $\mathcal{T}(P) = \{t_1, t_3, t_5\}$. OccurrenceDeliver duyệt ba giao dịch này đúng một lần. Với mỗi giao dịch, nhìn vào các item nó chứa (ngoài A) và rải TID của giao dịch đó vào bucket của từng item tương ứng, như minh họa bên cạnh.

Sau một lượt duyệt như vậy, ta có ngay $\text{Bucket}[e] = \mathcal{T}(P \cup \{e\})$ cho cả ba item B, C, D cùng lúc, mà chỉ tốn một lượt duyệt, đúng như đã đếm ở slide trước.



Bất biến (invariant) sau khi deliver

Bất biến cần nhớ

Sau khi duyệt xong mọi $t \in \mathcal{T}(P)$ và rải vào bucket, đẳng thức sau đúng với mọi item e :

$$\text{Bucket}[e] = \{t \in \mathcal{T}(P) : e \in t\} = \mathcal{T}(P \cup \{e\}).$$

Nói cách khác, bucket của e chính là denotation của $P \cup \{e\}$, không phải một cấu trúc tạm để minh họa. Vì vậy support của $P \cup \{e\}$ chỉ đơn giản là đếm số phần tử trong bucket đó: $\text{sup}(P \cup \{e\}) = |\text{Bucket}[e]|$.

Đây là điểm quan trọng nhất cần nhớ: bucket có thể dùng luôn cho bước tiếp theo (kiểm tra ngưỡng minsup, đệ quy, hoặc, như sẽ thấy ở phần HypercubeDecomposition, phát hiện item nào không làm giảm support), mà không cần tính lại bất cứ điều gì.

Một kỹ thuật khác để tăng tốc down project là dùng **bitmap** (biểu diễn denotation bằng dãy bit, giao bằng phép AND). Uno et al. (2004) nhận định:

Trích bài báo

“A 32bit CPU can take the intersection of 32bits at once, thus roughly speaking the computation time is reduced to 1/32. This method has a disadvantage for sparse datasets... bitmap method seems to be not good for sparse large datasets.”

Vì vậy LCMFreq nguyên gốc không dùng bitmap, mà dùng OccurrenceDeliver với mảng số nguyên đơn giản, hiệu quả cho cả dataset thưa và dày đặc. Bản cài đặt tối ưu của nhóm (trình bày ở phần Cài đặt Julia) lại chọn dùng BitArray, đúng như nhận định trên: kết quả thực nghiệm cho thấy mức tăng tốc lớn nhất rơi vào các dataset dày đặc (Chess, Pumsb), còn dataset thưa (Retail) tăng tốc khiêm tốn hơn nhiều.

Đặt vấn đề

Khung Backtracking

Đếm support hiệu quả

HypercubeDecomposition

Pseudo-code và ví dụ chạy tay

Cài đặt Julia

Đánh giá thực nghiệm

Ứng dụng: phân tích giỏ hàng

Kết luận

Tài liệu tham khảo

Định nghĩa (Uno et al., 2004, Mục 3.3)

Với itemset P , gọi $H(P)$ là tập các item $e > \text{tail}(P)$ sao cho $\mathcal{T}(P) = \mathcal{T}(P \cup \{e\})$. Khi đó, với mọi $Q \subseteq H(P)$, $\mathcal{T}(P \cup Q) = \mathcal{T}(P)$, nên $P \cup Q$ cũng frequent. Vì $\text{Bucket}[e] = \mathcal{T}(P \cup \{e\})$ đã có sẵn sau OccurrenceDeliver, việc kiểm tra $e \in H(P)$ chỉ cần so độ dài: $|\text{Bucket}[e]| = |\mathcal{T}(P)|$.

Xuất theo lô, theo đúng phát biểu của bài báo

Đặt $S' = S \cup H(P)$ (S là tập item miễn phí tích lũy từ node cha). Thuật toán xuất mọi itemset P' thỏa $P \subseteq P' \subseteq P \cup S'$ trong một lượt. Uno et al. (2004) khẳng định: “*This saves about $2^{|H(P)|}$ times of the frequency counting.*” Khi tạo nhánh con với item $e \notin S'$, S' được truyền xuống, và các item đã thuộc S' bị loại khỏi danh sách candidate để tránh sinh lại.

Lập luận

Theo định nghĩa, nếu $e \in H(P)$ thì $\mathcal{T}(P \cup \{e\}) = \mathcal{T}(P)$. Áp dụng liên tiếp cho từng item trong $Q \subseteq S' = S \cup H(P)$, denotation không hề thay đổi qua mỗi lần thêm, nên $\mathcal{T}(P \cup Q) = \mathcal{T}(P) \Rightarrow \text{sup}(P \cup Q) = \text{sup}(P)$. Điều này đúng với mọi tổ hợp con Q của S' , kể cả Q rỗng (chính là P) và $Q = S'$ (lấy hết mọi item miễn phí).

Hệ quả cho thuật toán:

- Tất cả $P \cup Q$ với $Q \subseteq S'$ đều là frequent và được xuất ngay tại node P , không cần đếm support riêng cho từng cái.
- Không cần đệ quy riêng vào các item thuộc S' , vì đệ quy vào chúng cũng chỉ ra đúng những itemset đã xuất rồi.
- Khi tạo nhánh con để đi tiếp, các item trong S' bị loại khỏi danh sách candidate, để tránh sinh lại đúng những gì đã xuất.

Trực giác chung

HypercubeDecomposition không bỏ sót và không thêm sai itemset nào. Nó chỉ gom một nhóm node có cùng denotation, nên cùng support, lại để xuất một lần, giúp giảm số nhánh phải đi qua mà vẫn cho ra đúng tập frequent itemset như khi đệ quy đầy đủ.

Đặt vấn đề

Khung Backtracking

Đếm support hiệu quả

HypercubeDecomposition

Pseudo-code và ví dụ chạy tay

Cài đặt Julia

Đánh giá thực nghiệm

Ứng dụng: phân tích giỏ hàng

Kết luận

Tài liệu tham khảo

```
LCMFreqBase(P, tids_P, F, S):
    support_P = length(tids_P)

    Bucket[e] = empty list for each e in F
    for tid in tids_P:
        for item in transaction[tid]:
            if item in F:
                push tid into Bucket[item]

    H = {e in F : length(Bucket[e]) == support_P}
    S_prime = S union H

    for Q subset of S_prime:
        output P union Q with support_P

    for e in F, e > tail(P), e not in S_prime:
        if length(Bucket[e]) >= minsup:
            F_next = frequent items after e
            LCMFreqBase(P union {e}, Bucket[e], F_next, S_prime)
```

Nhập chạy tại một node P :

1. Dựng bucket bằng OccurrenceDeliver (dòng 4 đến 6): duyệt $\mathcal{T}(P)$ một lần.
2. Suy ra $H(P)$ từ độ dài bucket, gộp vào S' (dòng 8 và 9).
3. Xuất hàng loạt $P \cup Q$, $Q \subseteq S'$ (dòng 11 và 12).
4. Đệ quy tiếp vào item ngoài S' , còn đủ minsup (dòng 14 đến 17).

Ví dụ chạy tay: database và bước gốc

TID	Items
T_1	A, B, C, D
T_2	A, B, D
T_3	A, B, C
T_4	B, C
T_5	A, B, C, D

Ngưỡng $\sigma = 2$. Bắt đầu từ prefix rỗng $P = \emptyset$, khi đó $\mathcal{T}(\emptyset)$ là toàn bộ 5 giao dịch.

Chạy OccurrenceDeliver tại root: duyệt cả 5 giao dịch một lần, rải từng giao dịch vào bucket của mọi item nó chứa:

e	Bucket[e]	sup
A	T_1, T_2, T_3, T_5	4
B	T_1, T_2, T_3, T_4, T_5	5
C	T_1, T_3, T_4, T_5	4
D	T_1, T_2, T_5	3

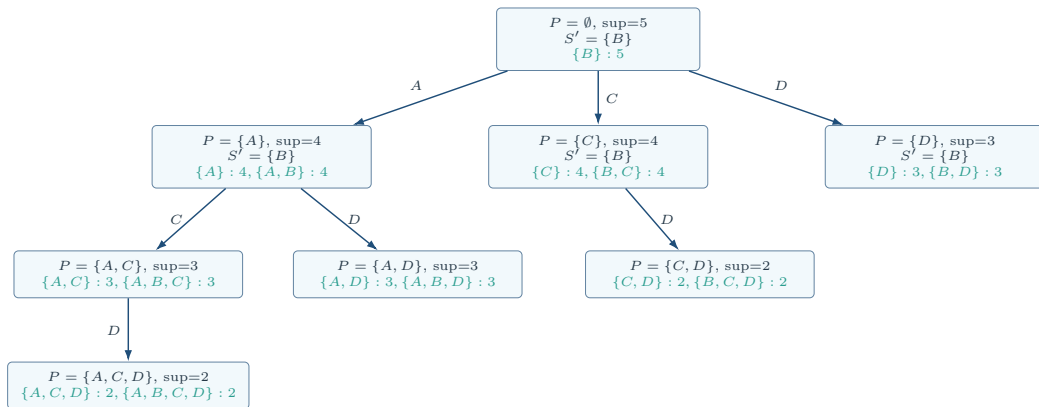
Nhìn vào bảng kết quả: $|\text{Bucket}[B]| = 5$, bằng đúng $|\mathcal{T}(\emptyset)| = 5$.

Điều đó nghĩa là B xuất hiện trong mọi giao dịch, thêm B vào không loại bỏ giao dịch nào, nên B là item miễn phí: $B \in H(\emptyset)$. Ba item còn lại (A, C, D) có bucket ngắn hơn 5, nên chưa miễn phí.

Ta gộp $S' = \{B\}$ và xuất ngay, không cần đệ quy: $\{B\} : 5$.

Sau đó chỉ đệ quy tiếp với A, C, D , các item còn lại, chưa miễn phí. Không đệ quy riêng cho B vì đã xuất xong ở trên. Mọi itemset chứa B sẽ tiếp tục được ghép kèm tự động ở các node con, như sẽ thấy trong cây đệ quy ở slide sau.

Cây đệ quy trên ví dụ



Toàn bộ cây DFS chỉ có 7 node thật sự, ứng với $\emptyset, A, C, D, AC, AD, CD, ACD$. Vì B luôn miễn phí ở mọi node (B xuất hiện trong mọi giao dịch), 8 itemset còn lại chứa B (như $\{B\}, \{A, B\}, \{A, B, C, D\}$) đều được xuất kèm theo bằng hypercube ngay tại node tương ứng, không cần tạo thêm một nhánh đệ quy riêng cho B .

Mục lục

Đặt vấn đề

Khung Backtracking

Đếm support hiệu quả

HypercubeDecomposition

Pseudo-code và ví dụ chạy tay

Cài đặt Julia

Đánh giá thực nghiệm

Ứng dụng: phân tích giỏ hàng

Kết luận

Tài liệu tham khảo

Code Julia baseline: ánh xạ từng khối

```
support_P = length(tids_P)

local_buckets = Dict{Int, Vector{Int}}{
    item => Int[] for item in freq_items
}

for tid in tids_P
    for item in transactions[tid]
        haskey(local_buckets, item) &&
            push!(local_buckets[item], tid)
    end
end
```

Đây chính là OccurrenceDeliver viết bằng Julia. Với mỗi transaction trong $\mathcal{T}(P)$ (`tids_P`), ta duyệt từng item của nó, và nếu item đó còn là candidate (`haskey`) thì đẩy TID vào bucket của item đó.

```
H_P = filter(e ->
    length(local_buckets[e]) == support_P,
    freq_items
)

S_prime = sort!(collect(union(S, H_P)))
S_prime_set = Set(S_prime)
```

Đây là HypercubeDecomposition. `filter` giữ lại đúng những item e có độ dài bucket bằng `support_P`, nghĩa là thêm e không làm mất giao dịch nào. Những item này (H_P) được gộp với S tích lũy từ node cha thành S_{prime} .

Code Julia baseline: xuất kết quả và đệ quy

```
for Q in _all_subsets_base(S_prime)
    (plen == 0 && isempty(Q)) && continue
    push!(
        result,
        FreqItemset(
            sort!([p[1:plen]; Q]),
            support_P
        )
    )
end
```

Đoạn này sinh mọi tập con Q của S_{prime} (`_all_subsets_base`) và xuất itemset $P \cup Q$ với support bằng `support_P`, chính là bước xuất theo lô của HypercubeDecomposition, làm một lần cho toàn bộ $2^{|S'|}$ tổ hợp.

```
for (j, e) in enumerate(freq_items)
    e <= tail_P && continue
    e in S_prime_set && continue

    tids_Pe = local_buckets[e]
    length(tids_Pe) < minsup && continue

    p[plen + 1] = e
    _hypercube_base!(
        p, plen + 1, tids_Pe,
        new_freq_items, transactions,
        minsup, result, S_prime
    )
end
```

Vòng lặp này quyết định đệ quy tiếp vào item nào: bỏ qua item nhỏ hơn hoặc bằng `tail(P)`, bỏ qua item đã thuộc S_{prime} , và bỏ qua item không đủ `minsup`. Với item còn lại, gọi đệ quy với bucket của nó làm denotation mới.

Tối ưu hóa bằng BitArray

Khung thuật toán (Backtracking, OccurrenceDeliver, HypercubeDecomposition) được giữ nguyên ở cả hai bản cài đặt. Khác biệt duy nhất là cách biểu diễn denotation và cách tính giao. Đây là tối ưu hóa riêng của nhóm, dựa trên nhận định của Uno et al. (2004) về bitmap đã trình bày ở phần trước.

Baseline: bám sát bài báo

- Denotation lưu dưới dạng `Vector<Int>`, một danh sách số nguyên các TID, giống cách bài báo mô tả.
- OccurrenceDeliver thực hiện đúng như đã trình bày: duyệt transaction, đẩy TID vào `Vector` của bucket tương ứng.
- Đơn giản, dễ kiểm chứng, nhưng *push!* vào vector tốn thời gian cấp phát bộ nhớ khi dữ liệu lớn.

Optimized: dùng BitArray

- Denotation đổi sang `BitArray{1}`: một chuỗi bit, bit thứ i bằng 1 nếu giao dịch i thuộc denotation.
- Giao hai denotation chỉ còn là phép AND theo bit, xử lý nhiều bit cùng lúc trong một lệnh CPU.
- Đếm support là đếm số bit 1 (`count`), tận dụng lệnh phần cứng POPCNT, rất nhanh so với duyệt từng phần tử.

```
tidset_Pe = tidset_P .& tidsets_full[e],    support_Pe = count(tidset_Pe).
```

Ý tưởng HypercubeDecomposition không đổi: vẫn tìm $H(P)$ bằng cách kiểm tra phép AND bitset của e với $\mathcal{T}(P)$ có giữ nguyên đúng số bit 1 hay không, chỉ là phép kiểm tra này giờ nhanh hơn nhiều.

Đặt vấn đề

Khung Backtracking

Đếm support hiệu quả

HypercubeDecomposition

Pseudo-code và ví dụ chạy tay

Cài đặt Julia

Đánh giá thực nghiệm

Ứng dụng: phân tích giỏ hàng

Kết luận

Tài liệu tham khảo

Thiết lập thực nghiệm

Thực nghiệm dùng 9 dataset chuẩn từ kho FIMI, đối chiếu với SPMF (Java) trên cùng máy, mỗi cấu hình lặp lại nhiều lần và lấy thời gian trung vị.

Dataset	Số transaction	Số item	Độ dài TB	Đặc điểm
Chess	3 196	75	37.0	dày đặc, itemset dài
Connect	67 557	129	43.0	dày đặc, rất nén
Mushroom	8 124	119	23.0	dày đặc, nhiều item
Pumsb	49 046	2 113	74.0	dày đặc, độ dài rất lớn
Accidents	340 183	468	33.8	rất lớn, trung gian
Retail	88 162	16 470	10.3	thưa, dữ liệu thực
T10I4D100K	100 000	870	10.1	tổng hợp, thưa
T40I10D100K	100 000	942	39.6	tổng hợp, thưa, dài
Kosarak	990 002	41 270	8.1	cực lớn, rất thưa

Chi phí tại một node P

Vì OccurrenceDeliver chỉ duyệt mỗi giao dịch trong $\mathcal{T}(P)$ đúng một lần, và với mỗi giao dịch chỉ xét các item còn là candidate F , chi phí của một node là $O\left(\sum_{T \in \mathcal{T}(P)} |T \cap F|\right)$, không có chuyện duyệt lại nhiều lần như cách down project.

Toàn thuật toán

Dù mỗi node rẻ hơn, tổng thời gian vẫn có thể tăng theo hàm mũ trong trường hợp xấu nhất, vì số frequent itemset bản thân nó đã có thể tăng theo hàm mũ theo $|\mathcal{I}|$. LCMFreq là thuật toán *output sensitive*: thời gian chạy gắn chặt với số lượng kết quả phải xuất ra, nên khi giảm minsup, số itemset tăng nhanh thì thời gian chạy cũng tăng nhanh theo.

Yếu tố tăng	Vì sao ảnh hưởng đến thời gian chạy
Số giao dịch $ \mathcal{D} $	mỗi node phải duyệt nhiều giao dịch hơn, hoặc AND nhiều bit hơn
Số item $ \mathcal{I} $	nhiều candidate hơn ở mỗi node, nên nhiều nhánh đệ quy hơn
Minsup thấp	ít itemset bị cắt tĩa, cây tìm kiếm và số kết quả nở rộng nhanh
Giao dịch dài	nhiều item đồng xuất hiện, nên nhiều itemset tiềm năng hơn
Dataset dày đặc	nhiều item có cùng denotation, Hypercube và BitArray phát huy tối đa

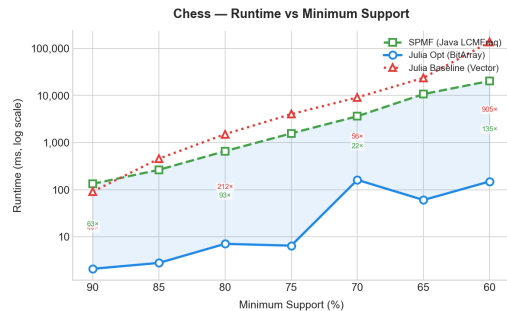
Đối chiếu số lượng frequent itemset với SPMF

Dataset	minsup	Julia	SPMF
Chess	75%	20 993	20 993
Connect	93%	7 203	7 203
Mushroom	30%	2 735	2 735
Pumsb	90%	2 607	2 607
Accidents	80%	149	149
Retail	2%	55	55
T10I4D100K	0.8%	494	494
T40I10D100K	2%	2 293	2 293
Kosarak	0.8%	583	583

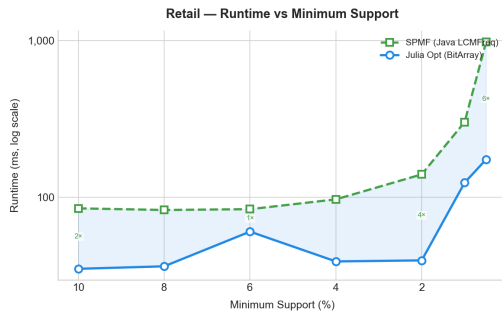
Kết quả khớp tuyệt đối 100% trên toàn bộ 9 dataset, không thiếu, không thừa, không sai support của bất kỳ itemset nào.

Ngoài đối chiếu số lượng, bộ unit test của nhóm còn so sánh trực tiếp giá trị support của từng itemset trên các toy dataset nhỏ (trong đó có ví dụ chạy tay đã trình bày ở phần trước), đảm bảo không chỉ đúng về số lượng mà còn đúng về nội dung của từng kết quả.

Thời gian chạy theo ngưỡng minsup



Chess (dày đặc)



Retail (thưa)

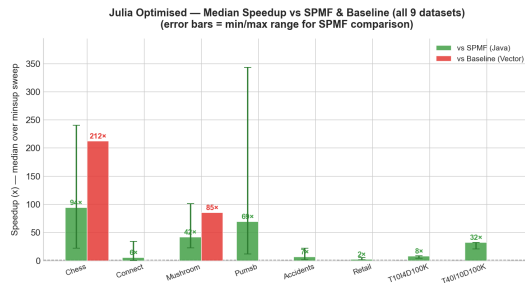
Trục đứng là thời gian chạy theo thang logarithm. Trên dataset dày đặc như Chess, bản tối ưu nhanh hơn SPMF từ 76 đến 141 lần. Trên dataset thưa như Retail, mức tăng tốc khiêm tốn hơn (khoảng 2 đến 6 lần), vì denotation vốn đã nhỏ nên down project không tốn nhiều như trên dataset dày đặc. Baseline (vector số nguyên) hết bộ nhớ (OOM) trên Retail do cấp phát theo chỉ số item lớn nhất, trong khi Retail có tới 16 470 item.

Tăng tốc so với SPMF: tổng quan

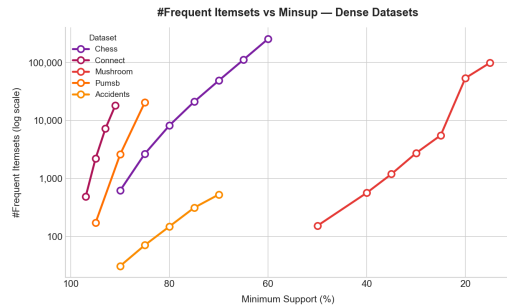
Dataset	Trung vị	Min	Max
Chess	135×	22×	241×
Mushroom	58×	23×	101×
Connect	6×	1×	34×
Pumsb	343×	12×	343×
Accidents	7×	2×	22×
Retail	2×	1×	6×
T10I4D100K	8×	4×	9×
T40I10D100K	32×	21×	32×

Kosarak: SPMF không chạy được trên máy thực nghiệm, nên không có số liệu so sánh trực tiếp.

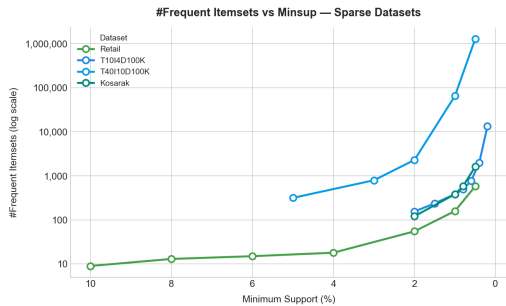
Pumsb đạt mức tăng tốc cao nhất nhờ độ dài giao dịch trung bình lớn (74 item), khiến lợi thế của phép AND bit trên BitArray được khuếch đại.



Số lượng frequent itemset theo minsup



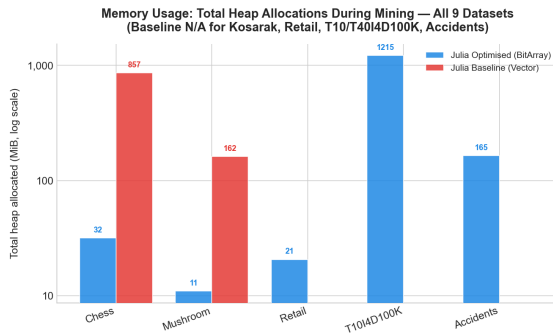
Dataset dày đặc



Dataset thưa

Khi σ giảm, ngưỡng chấp nhận thấp hơn, cây tìm kiếm ít bị cắt tỉa hơn, nên số frequent itemset tăng theo dạng hàm mũ trên dataset dày đặc. Trên dataset thưa, số lượng itemset tăng chậm hơn nhiều, vì ít item đồng xuất hiện đủ thường xuyên để vượt ngưỡng. Đây là minh chứng thực nghiệm trực tiếp cho tính chất đơn điệu đã trình bày ở phần Đặt vấn đề.

Dataset	Opt (MiB)	Baseline	Giảm
Chess	31.87	857.23	26.9×
Mushroom	10.98	161.57	14.7×
Accidents	165.16	OOM	N/A
Retail	20.68	OOM	N/A
T10I4D100K	1 215.2	OOM	N/A

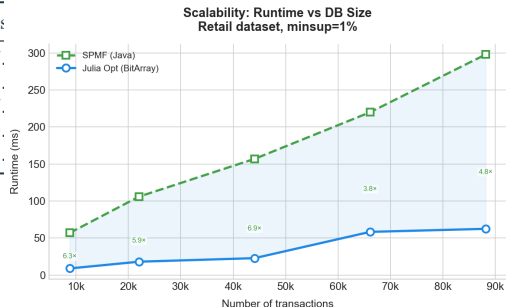


Baseline dùng `Vector{Int}` cho denotation, mỗi phần tử chiếm 8 byte. BitArray chỉ chiếm 1 bit mỗi transaction, tiết kiệm khoảng $64\times$ trên mỗi denotation. Baseline còn cấp phát mảng theo chỉ số item lớn nhất, gây lãng phí nghiêm trọng khi item ID thưa, ví dụ Retail có chỉ số item lớn nhất bằng 16 470. Vì vậy baseline hết bộ nhớ trên 7 trong 9 dataset. Bản tối ưu dùng cấu trúc kiểu từ điển, chỉ cấp phát cho item thực sự frequent, nên khắc phục hoàn toàn vấn đề này.

Khả năng mở rộng theo kích thước dữ liệu

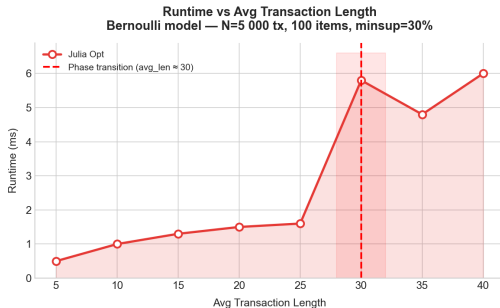
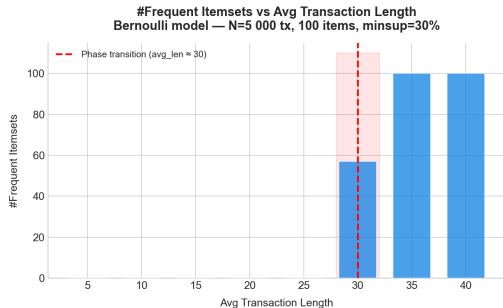
Tỉ lệ	Số transaction	Julia Opt (ms)	SPMF (ms)
10%	8 816	9.0	57.
25%	22 040	18.1	106.
50%	44 081	22.9	157.
75%	66 122	58.4	220.
100%	88 162	62.5	298.

Lấy mẫu ngẫu nhiên không lặp từ Retail, minsup tuyệt đối tăng tỉ lệ với kích thước tập con, để giữ nguyên độ khó bài toán.



Thời gian chạy tăng gần tuyến tính theo số transaction, đúng như lý thuyết: mỗi phép AND trên BitArray tốn thêm chi phí tỉ lệ với n khi n tăng, còn số lượng itemset phổ biến giữ ổn định vì minsup tương đối không đổi.

Ảnh hưởng của độ dài giao dịch



Dataset tổng hợp với 5 000 giao dịch, 100 item, minsup = 30% (ngưỡng tuyệt đối là 1 500). Khi độ dài giao dịch trung bình nhỏ hơn 25, mỗi item chỉ xuất hiện trong khoảng 1 250 giao dịch, chưa đủ ngưỡng, nên kết quả rỗng. Khi độ dài trung bình đạt khoảng 30, support kỳ vọng vượt ngưỡng và các itemset phổ biến bắt đầu xuất hiện đột ngột. Đây là ngưỡng pha chuyển (phase transition) của bài toán Frequent Itemset Mining.

Đặt vấn đề

Khung Backtracking

Đếm support hiệu quả

HypercubeDecomposition

Pseudo-code và ví dụ chạy tay

Cài đặt Julia

Đánh giá thực nghiệm

Ứng dụng: phân tích giỏ hàng

Kết luận

Tài liệu tham khảo

Từ frequent itemset đến association rule

Với mỗi itemset frequent X và mỗi tập con khác rỗng $Y \subset X$, ta xét luật $Y \Rightarrow (X \setminus Y)$. Luật được giữ lại nếu độ tin cậy đạt ngưỡng tối thiểu:

$$\text{conf}(Y \Rightarrow X \setminus Y) = \frac{\text{sup}(X)}{\text{sup}(Y)} \geq \text{minconf}.$$

Chỉ số lift cho biết mức tương quan so với việc hai vế độc lập: $\text{lift} > 1$ nghĩa là chúng có xu hướng đồng xuất hiện nhiều hơn ngẫu nhiên.

Thiết lập thực nghiệm trên dataset Retail (88 162 giao dịch): $\text{minsup} = 0.5\%$ ($\sigma = 441$), thu được 580 frequent itemset; với $\text{minconf} = 60\%$, sinh được 312 association rule.

Top luật kết hợp theo lift trên Retail

Vế trái	Vế phải	Support	Confidence	Lift
{38, 39}	{41}	1.01%	0.89	8.7
{38}	{39, 41}	1.01%	0.82	8.1
{39, 41}	{38}	1.01%	0.80	7.9
{38, 41}	{39}	1.01%	0.78	7.7
{39}	{38}	1.18%	0.73	7.2
{32}	{48}	0.72%	0.78	6.4

Nhóm sản phẩm {38, 39, 41} có lift trên 7, cho thấy xác suất đồng xuất hiện cao gấp khoảng 7 lần so với ngẫu nhiên, là candidate mạnh cho chiến lược bán theo gói hoặc bố trí gần nhau trên kệ. Dataset Retail mã hóa sản phẩm bằng số nguyên, không có nhãn tên sản phẩm trong dữ liệu công khai, nên phân tích chỉ dừng ở mức item ID.

Mục lục

Đặt vấn đề

Khung Backtracking

Đếm support hiệu quả

HypercubeDecomposition

Pseudo-code và ví dụ chạy tay

Cài đặt Julia

Đánh giá thực nghiệm

Ứng dụng: phân tích giỏ hàng

Kết luận

Tài liệu tham khảo

Đã hoàn thành

- Cài đặt đầy đủ Backtracking, OccurrenceDeliver, HypercubeDecomposition theo đúng Uno et al. (2004).
- Bản tối ưu BitArray, kiểm thử đơn vị tự động.
- Khớp 100% với SPMF trên toàn bộ 9 dataset.
- Ứng dụng phân tích giỏ hàng trên Retail.

Giới hạn

- Bộ nhớ còn cao trên T10I4D100K (khoảng 1.2 GiB).
- Mỗi denotation vẫn lưu toàn bộ BitArray, chưa dùng diffset.
- Chưa khai thác đa luồng cho các nhánh DFS độc lập.

Hướng phát triển

- Áp dụng diffset để giảm bộ nhớ khi support gần với node cha.
- Song song hóa các nhánh top level bằng đa luồng Julia.
- Mở rộng sang khai thác closed và maximal itemset (LCM gốc).

Tóm lại

LCMFreq dùng khung backtracking để tiết kiệm bộ nhớ so với Apriori, rồi khắc phục điểm yếu của backtracking trong việc đếm support bằng OccurrenceDeliver, và giảm số lần đếm lặp lại bằng HypercubeDecomposition. Bản baseline của nhóm bám sát mô tả của bài báo gốc, không dùng thư viện FIM có sẵn. Bản tối ưu giữ nguyên khung sinh itemset đó, chỉ thay cấu trúc denotation bằng BitArray để tăng tốc phép giao và đếm support, là đóng góp tối ưu hóa riêng của nhóm, không phải sao chép từ SPMF. SPMF chỉ được dùng làm chuẩn đối chiếu kết quả.

Mục lục

Đặt vấn đề

Khung Backtracking

Đếm support hiệu quả

HypercubeDecomposition

Pseudo-code và ví dụ chạy tay

Cài đặt Julia

Đánh giá thực nghiệm

Ứng dụng: phân tích giỏ hàng

Kết luận

Tài liệu tham khảo

1. T. Uno, M. Kiyomi, H. Arimura. *LCM ver.2: Efficient Mining Algorithms for Frequent/Closed/Maximal Itemsets*. IEEE ICDM Workshop on Frequent Itemset Mining Implementations (FIMI), Brighton, UK, 2004.
2. R. Agrawal, R. Srikant. *Fast Algorithms for Mining Association Rules*. Proc. 20th VLDB, pp. 487–499, Santiago, Chile, 1994.
3. J. Han, J. Pei, Y. Yin. *Mining Frequent Patterns without Candidate Generation*. Proc. ACM SIGMOD, pp. 1–12, Dallas, TX, 2000.
4. M. J. Zaki, S. Parthasarathy, M. Ogihara, W. Li. *New Algorithms for Fast Discovery of Association Rules*. Proc. 3rd KDD, pp. 283–286, Newport Beach, CA, 1997.